BESFEM: Explaining the Math and Code

1 Lithium Concentration Governing Equations

1.1 Cathode Lithium Concentration (Smoothed Boundary Diffusion)

Lithium transport in the cathode is governed by diffusion inside the particle and electrochemical reactions at the particle-electrolyte interface. The smoothed boundary method (SBM) embeds both effects in a single equation by using a phase indicator field ψ_c that smoothly transitions between cathode and non-cathode regions.

1.1.1 Strong form with smoothed boundary weighting

The lithium concentration in the cathode is:

$$\frac{\partial C_c}{\partial t} = \frac{1}{\psi_c}\nabla \cdot \left(\psi_c D_c \nabla C_c\right) \; - \; \frac{|\nabla \psi_c|}{\psi_c}\, r_c \tag{1}$$

1.1.2 Step 1: Multiplying Eq. (1) by ψ_c

$$\psi_c \frac{\partial C_c}{\partial t} = \nabla \cdot (\psi_c D_c \nabla C_c) - |\nabla \psi_c| r_c \tag{2}$$

1.1.3 Step 2: Weak form

Multiplying Eq. (2) by test function v and integrating over Ω gives

$$\int_\Omega \underbrace{\psi_c v \frac{\partial C_c}{\partial t}}_{\text{Time term}} \, d\Omega = \underbrace{\int_\Omega v \nabla \cdot (\psi_c D_c \nabla C_c)\, d\Omega}_{\text{Diffusion}} - \int_\Omega \underbrace{v |\nabla \psi_c| r_c}_{\text{Reaction}} \, d\Omega \tag{3}$$

Applying integration by parts to the diffusion term:

$$\int_\Omega v \nabla \cdot (\psi_c D_c \nabla C_c)\, d\Omega = -\int_\Omega \psi_c D_c \nabla C_c \cdot \nabla v \, d\Omega \tag{4}$$

1.1.4 Final weak form

The weak form solved in BESFEM is

$$\boxed{\int_\Omega \psi_c v \frac{\partial C_c}{\partial t}\, d\Omega + \int_\Omega \psi_c D_c \nabla C_c \cdot \nabla v\, d\Omega = -\int_\Omega v |\nabla \psi_c| r_c\, d\Omega} \tag{5}$$

1.1.5 Finite-element discretization

Using basis functions $\{\phi_i\}$:

$$\underbrace{M\dot{\mathbf{C}}}_{\text{Transient}} + \underbrace{K\mathbf{C}}_{\text{Diffusion}} = \underbrace{\mathbf{F}}_{\text{Reaction}} \tag{6}$$

with

$$M_{ij} = \int_\Omega \psi_c \phi_i \phi_j\, d\Omega \tag{7}$$

$$K_{ij} = \int_\Omega \psi_c D_c \nabla \phi_i \cdot \nabla \phi_j\, d\Omega \tag{8}$$

$$F_i = -\int_\Omega \phi_i |\nabla \psi_c| r_c\, d\Omega \tag{9}$$

1.1.6 M, K, and F in the code

Mass matrix M (transient term)

The mass matrix is assembled once during setup.

$$M_{ij} = \int_\Omega \psi_c \phi_i \phi_j\, d\Omega$$

In the code, the cathode indicator field ψ_c is stored in the grid function `psx`.

```
mfem::GridFunctionCoefficient coef(&psx);
fem.InitializeMassMatrix(coef, Mt);
fem.FormSystemMatrix(Mt, boundary_dofs, Mmatp);
```

This produces the boundary-eliminated mass matrix

$$\boxed{M \equiv \texttt{Mmatp}}$$

Stiffness matrix K (diffusion term)

The stiffness matrix

$$K_{ij} = \int_\Omega \psi_c D_c \nabla \phi_i \cdot \nabla \phi_j\, d\Omega$$

depends on the diffusivity D_c, which depends on the current solution C_c. Therefore, K is updated at every time step.

First, the diffusivity is evaluated pointwise at the mesh vertices:

```
for (int vi = 0; vi < nV; vi++){
        Dp(vi) = psx(vi) * (0.0277 - 0.084 * Cn(vi)
        + 0.1003 * Cn(vi) * Cn(vi)) * 1.0e-8;
    }
cDp.SetGridFunction(&Dp);
```

Next, the stiffness bilinear form is updated and assembled:

```
fem.InitializeStiffnessMatrix(cDp, Kc2);
fem.FormLinearSystem(Kc2, boundary_dofs, Cn, Fct, Kmatp, X1v, Fcb);
```

This produces the stiffness matrix

$$\boxed{K \equiv \texttt{Kmatp}}$$

Reaction vector F

$$F_i = -\int_\Omega \phi_i |\nabla \psi_c| r_c \, d\Omega$$

Assembled via:

```
utils.InitializeReaction(Rx, RxC, (1.0/Constants::rho_C));
cAp.SetGridFunction(&RxC);
fem.InitializeForceTerm(cAp, Bc2);
fem.Update(Bc2);
Fct = *Bc2;
```

This produces the reaction vector

$\boxed{\mathbf{F} \equiv \texttt{Fct}}$ and after elimination via `FormLinearSystem`: $\boxed{\mathbf{F}_{\text{elim}} \equiv \texttt{Fcb}}$

1.1.7 Solving the Linear System

After the mass matrix M, stiffness matrix K, and reaction vector $\mathbf{F}$ have been assembled, the cathode concentration at the next timestep is obtained by solving a linear system.

The system being solved

After discretization and implicit Backward Euler time integration, the code solves the linear system

$$\boxed{(M - \Delta t\, K)\, \mathbf{C}^{n+1} = (M - \Delta t\, K)\, \mathbf{C}^n + \Delta t\, \mathbf{F}^{n+1}} \tag{10}$$

Right-hand side construction

Before solving the system, the right-hand side vector is assembled. First, the solution at the previous timestep is extracted:

```
Cn.GetTrueDofs(CpV0);
```

This corresponds to the vector $\mathbf{C}^n$.

Next, the Backward Euler operator is applied to the previous solution:

```
Tmatp.reset(Add(1.0, Mmatp, -Constants::dt, Kmatp));
```

$$\boxed{T \equiv \texttt{Tmatp} = (M - \Delta t K)}$$

```
Tmatp->Mult(CpV0, RHCp);
```

$$\texttt{RHCp} = T\mathbf{C}^n$$

Finally, the time-scaled reaction contribution is added:

```
Fcb *= Constants::dt;
RHCp += Fcb;
```

so that the complete right-hand side becomes

$$\boxed{\mathbf{b} \equiv RHCp = T\mathbf{C}^n + \Delta t\,\mathbf{F}^{n+1}}$$

Solving the linear system

The linear system:

$$T\,\mathbf{C}^{n+1} = \mathbf{b}$$

is solved by setting the operator and calling `Mult`:

```
Mp_solver.SetOperator(*Tmatp);
Mp_solver.Mult(RHCp, CpV0);
Cn.Distribute(CpV0);
```

So the line

$$\boxed{\texttt{Mp_solver.Mult(RHCp, CpV0)}}$$

is where the cathode concentration linear system is solved.

1.2 Electrolyte Lithium Concentration (Smoothed Boundary Diffusion)

Lithium transport in the electrolyte is governed by diffusion in the electrolyte region and by source terms at the cathode-electrolyte and anode-electrolyte interfaces. In the smoothed boundary method (SBM), the electrolyte indicator ψ_e weights the diffusion operator, while interfacial reactions enter as volumetric sources proportional to $|\nabla\psi_c|$ and $|\nabla\psi_a|$.

1.2.1 Strong form with smoothed boundary weighting

The electrolyte concentration is:

$$\frac{\partial C_e}{\partial t} = \frac{1}{\psi_e}\nabla\cdot\left(\psi_e D_e \nabla C_e\right) + \frac{|\nabla\psi_c|}{\psi_e}\frac{r_c t_-}{\nu_+} + \frac{|\nabla\psi_a|}{\psi_e}\frac{r_a t_-}{\nu_+} \tag{11}$$

Here, D_e is the ambipolar diffusivity of the salt, t_- is the transference number, and ν_+ is the dissolution number (cation).

1.2.2 Step 1: Multiplying Eq. (11) by ψ_e

$$\psi_e\frac{\partial C_e}{\partial t} = \nabla\cdot(\psi_e D_e\nabla C_e) + |\nabla\psi_c|\frac{r_c t_-}{\nu_+} + |\nabla\psi_a|\frac{r_a t_-}{\nu_+} \tag{12}$$

1.2.3 Step 2: Weak form

Multiply Eq. (12) by a test function v and integrate over Ω:

$$\int_\Omega \underbrace{\psi_e v\frac{\partial C_e}{\partial t}}_{\text{Time term}}\,d\Omega = \underbrace{\int_\Omega v\nabla\cdot(\psi_e D_e\nabla C_e)\,d\Omega}_{\text{Diffusion}} + \int_\Omega \underbrace{v|\nabla\psi_c|\frac{r_c t_-}{\nu_+}}_{\text{Cathode source}}\,d\Omega + \int_\Omega \underbrace{v|\nabla\psi_a|\frac{r_a t_-}{\nu_+}}_{\text{Anode source}}\,d\Omega \tag{13}$$

Apply integration by parts to the diffusion term:

$$\int_\Omega v\nabla\cdot(\psi_e D_e\nabla C_e)\,d\Omega = -\int_\Omega \psi_e D_e\nabla C_e\cdot\nabla v\,d\Omega \tag{14}$$

1.2.4 Final weak form

The weak form solved in BESFEM for the electrolyte is

$$\boxed{\int_\Omega \psi_e v\frac{\partial C_e}{\partial t}\,d\Omega + \int_\Omega \psi_e D_e\nabla C_e\cdot\nabla v\,d\Omega = \int_\Omega v|\nabla\psi_c|\frac{r_c t_-}{\nu_+}\,d\Omega + \int_\Omega v|\nabla\psi_a|\frac{r_a t_-}{\nu_+}\,d\Omega} \tag{15}$$

1.2.5 Finite-element discretization

Using basis functions $\{\phi_i\}$:

$$\underbrace{M\dot{\mathbf{C}}}_{\text{Transient}} + \underbrace{K\mathbf{C}}_{\text{Diffusion}} = \underbrace{\mathbf{F}}_{\text{Interface sources}} \tag{16}$$

with

$$M_{ij} = \int_\Omega \psi_e \phi_i \phi_j \, d\Omega \tag{17}$$

$$K_{ij} = \int_\Omega \psi_e D_e \nabla\phi_i \cdot \nabla\phi_j \, d\Omega \tag{18}$$

$$F_i = \int_\Omega \phi_i \left(|\nabla\psi_c| \frac{r_c t_-}{\nu_+} + |\nabla\psi_a| \frac{r_a t_-}{\nu_+} \right) d\Omega \tag{19}$$

1.2.6 M, K, and F in the code (CnE)

Mass matrix M ***(transient term)***

The mass matrix is assembled once during setup:

$$M_{ij} = \int_\Omega \psi_e \phi_i \phi_j \, d\Omega$$

In the code, ψ_e is the grid function `psx`:

```
mfem::GridFunctionCoefficient coef(&psx);
fem.InitializeMassMatrix(coef, Me_init);
fem.FormSystemMatrix(Me_init, boundary_dofs, Mmate);
```

This produces the mass matrix

$$\boxed{M \equiv \texttt{Mmate}}$$

Stiffness matrix K ***(diffusion term)***

The stiffness matrix

$$K_{ij} = \int_\Omega \psi_e D_e \nabla\phi_i \cdot \nabla\phi_j \, d\Omega$$

depends on the diffusivity D_e, which depends on the current solution C_e. Therefore, K is updated at every time step.

```
for (int vi = 0; vi < nV; vi++){
    De(vi) = psx(vi)*Constants::D0*exp(-7.02-830*Cn(vi)+50000*Cn(vi)*Cn(vi));
}
cDe.SetGridFunction(&De);
```

Next, the stiffness bilinear form is updated and assembled:

```
fem.InitializeStiffnessMatrix(cDe, Ke2);
fem.FormLinearSystem(Ke2, boundary_dofs, Cn, Fet, Kmate, X1v, Feb);
```

This produces the stiffness matrix

$$\boxed{K \equiv \texttt{Kmate}}$$

Reaction vector F

The reaction vector is:

$$F_i = \int_\Omega \phi_i \left(|\nabla \psi_c| \frac{r_c t_-}{\nu_+} + |\nabla \psi_a| \frac{r_a t_-}{\nu_+} \right) d\Omega$$

The electrolyte source comes from reactions at both interfaces (cathode-electrolyte and anode-electrolyte). In the code, those effects are combined into a single grid function `Rxe` (via `utils.InitializeReaction`) and then assembled with `DomainLFIntegrator(cAe)`. Here the source represents salt production/consumption in the electrolyte due to both electrodes (scaled by `t_minus`).

```
utils.InitializeReaction(RxA, RxC, Rxe, (-1.0 * Constants::t_minus));
cAe.SetGridFunction(&Rxe);

fem.InitializeForceTerm(cAe, Be_init);
fem.Update(Be_init);
Fet = *Be_init;
```

This produces the reaction vector:

$\boxed{\mathbf{F} \equiv \texttt{Fet}}$ and after elimination via `FormLinearSystem`: $\boxed{\mathbf{F}_{\text{elim}} \equiv \texttt{Feb}}$

1.2.7 Solving the Linear System

The electrolyte update uses **Crank-Nicolson**:

$$\boxed{\left(M - \tfrac{\Delta t}{2} K\right) \mathbf{C}^{n+1} = \left(M + \tfrac{\Delta t}{2} K\right) \mathbf{C}^n + \Delta t\, \mathbf{F}^{n+\frac{1}{2}}}$$

Crank-Nicolson in the code

```
TmatL.reset(Add(1.0, Mmate, -0.5 * Constants::dt, Kmate));
TmatR.reset(Add(1.0, Mmate,  0.5 * Constants::dt, Kmate));
```

So,

$$\boxed{T_L \equiv \texttt{TmatL} = (M - \tfrac{\Delta t}{2} K)} \quad \text{and} \quad \boxed{T_R \equiv \texttt{TmatR} = (M + \tfrac{\Delta t}{2} K)}$$

Right-hand side construction

```
Cn.GetTrueDofs(CeV0);
TmatR->Mult(CeV0, RHCe);
RHCe += Feb;
```

Thus,

$$\boxed{\mathbf{b} \equiv \texttt{RHCe} = T_R\, \mathbf{C}^n + \Delta t\, \mathbf{F}}$$

Solving the linear system

The linear system:

$$T_L \, \mathbf{C}^{n+1} = \mathbf{b}$$

is solved by setting the operator and calling `Mult`:

```
Me_solver.SetOperator(*TmatL);
Me_solver.Mult(RHCe, CeV0);
Cn.Distribute(CeV0);
```

So the line

```
Me_solver.Mult(RHCe, CeV0)
```

is where the electrolyte concentration linear system is solved.

1.3 Anode Lithium Concentration (Cahn-Hilliard with Smoothed Boundary)

Lithium transport in the anode is governed by phase-separating thermodynamics rather than diffusion due to the material properties of graphite. This behavior is modeled using a Cahn-Hilliard equation, which evolves the concentration C_a according to gradients in the chemical potential. The smoothed boundary method (SBM) embeds the anode-electrolyte interface via the indicator function ψ_a, while interfacial electrochemical reactions enter as volumetric sink terms.

1.3.1 Strong form with smoothed boundary weighting

The lithium concentration in the anode is:

$$\frac{\partial C_a}{\partial t} = \frac{1}{\psi_a}\nabla \cdot \left[\psi_a M_a \nabla \left(\frac{\partial f_G}{\partial C_a} - \varepsilon \nabla^2 C_a\right)\right] - \frac{|\nabla \psi_a|}{\psi_a} r_a \tag{20}$$

Here, $f_G(C_a)$ is the free-energy density of the anode material, ε is the gradient-energy coefficient controlling the diffuse phase boundary width, $M_a(C_a)$ is the concentration-dependent mobility, and r_a is the Butler-Volmer reaction rate at the anode-electrolyte interface.

1.3.2 Step 1: Multiplying Eq. (20) by ψ_a

$$\psi_a \frac{\partial C_a}{\partial t} = \nabla \cdot \left[\psi_a M_a \nabla \left(\frac{\partial f_G}{\partial C_a} - \varepsilon \nabla^2 C_a\right)\right] - |\nabla \psi_a| r_a \tag{21}$$

1.3.3 Step 2: Weak form

Multiplying Eq. (21) by a test function v and integrating over Ω gives

$$\begin{aligned}\int_\Omega \underbrace{\psi_a v \frac{\partial C_a}{\partial t}}_{\text{Time term}} \, d\Omega = &\int_\Omega \underbrace{v \nabla \cdot \left[\psi_a M_a \nabla \left(\frac{\partial f_G}{\partial C_a} - \varepsilon \nabla^2 C_a\right)\right]}_{\text{Cahn–Hilliard}} \, d\Omega \\ &- \int_\Omega \underbrace{v |\nabla \psi_a| r_a}_{\text{Reaction}} \, d\Omega.\end{aligned} \tag{22}$$

1.3.4 Final weak form

After integration by parts, the weak form solved in BESFEM for the anode is

$$\boxed{\int_\Omega \psi_a v \frac{\partial C_a}{\partial t} \, d\Omega + \int_\Omega \psi_a M_a \nabla \left(\frac{\partial f_G}{\partial C_a} - \varepsilon \nabla^2 C_a\right) \cdot \nabla v \, d\Omega = - \int_\Omega v |\nabla \psi_a| r_a \, d\Omega} \tag{23}$$

1.3.5 Finite-element discretization

Using basis functions $\{\phi_i\}$, the weak form is discretized as

$$\underbrace{M\dot{\mathbf{C}}}_{\text{Transient}} + \underbrace{K_M\left(\boldsymbol{\mu}_b - K_\varepsilon \mathbf{C}\right)}_{\text{Cahn–Hilliard flux}} = \underbrace{\mathbf{F}}_{\text{Reaction}} \tag{24}$$

with

$$M_{ij} = \int_\Omega \psi_a \phi_i \phi_j \, d\Omega, \tag{25}$$

$$(K_\varepsilon)_{ij} = \int_\Omega \varepsilon \nabla\phi_i \cdot \nabla\phi_j \, d\Omega, \tag{26}$$

$$(K_M)_{ij} = \int_\Omega \psi_a M_a \nabla\phi_i \cdot \nabla\phi_j \, d\Omega, \tag{27}$$

$$F_i = -\int_\Omega \phi_i |\nabla\psi_a| r_a \, d\Omega. \tag{28}$$

Here, $\boldsymbol{\mu}_b$ denotes $\partial f_G / \partial C_a$ evaluated from tabulated thermodynamic data.

1.3.6 M, K_ε, K_M, and F in the code (CnA)

Mass matrix M ***(transient term)***

$$M_{ij} = \int_\Omega \psi_a \phi_i \phi_j \, d\Omega$$

In the code, ψ_a is stored in the grid function `psx`:

```
mfem::GridFunctionCoefficient coef(&psx);
fem.InitializeMassMatrix(coef, M_init);
fem.FormSystemMatrix(M_init, boundary_dofs, MmatCH);
```

This produces the mass matrix

$$\boxed{M \equiv \texttt{MmatCH}}$$

Free-energy stiffness matrix K_E

The gradient-energy contribution $-\varepsilon\nabla^2 C_a$ is discretized as

$$(K_\varepsilon)_{ij} = \int_\Omega \varepsilon \nabla\phi_i \cdot \nabla\phi_j \, d\Omega$$

In the code:

```
mfem::ConstantCoefficient varE(Constants::gc /
        pow(Constants::dh, pmesh->Dimension()));
fem.InitializeStiffnessMatrix(varE, Grad_EForm);
fem.FormLinearSystem(Grad_EForm, boundary_dofs, Cn, Fct, Grad_EM, X1v, Fcb);
```

This produces the stiffness matrix

$$\boxed{K_\varepsilon \equiv \texttt{Grad_EM}}$$

Mobility stiffness matrix K_M

The mobility operator is

$$(K_M)_{ij} = \int_\Omega \psi_a M_a \nabla\phi_i \cdot \nabla\phi_j \, d\Omega$$

where $M_a(C_a)$ is concentration dependent and tabulated.

```
Mob(i) = psx(i) * GetTableValues(cn_val, Ticks, Mobility);
cDp.SetGridFunction(&Mob);
fem.InitializeStiffnessMatrix(cDp, Grad_MForm);
fem.Update(Grad_MForm);
fem.FormLinearSystem(Grad_MForm, boundary_dofs, Mub, Fct, Grad_MM, X1v, Fcb);
```

This produces the stiffness matrix

$$\boxed{K_M \equiv \texttt{Grad_MM}}$$

Reaction vector $\mathbf{F}$

$$F_i = -\int_\Omega \phi_i |\nabla\psi_a| r_a \, d\Omega$$

Assembled via:

```
utils.InitializeReaction(Rx, RxA, (1.0/Constants::rho_A));
cAp.SetGridFunction(&RxA);
fem.InitializeForceTerm(cAp, B_init);
fem.Update(B_init);
Fct = *B_init;
```

This produces the reaction vector:

$\boxed{\mathbf{F} \equiv \texttt{Fct}}$ and after elimination via `FormLinearSystem`: $\boxed{\mathbf{F}_{\text{elim}} \equiv \texttt{Fcb}}$

1.3.7 Solving the Linear System

The code uses Cahn-Hilliard to solve:

$$\boxed{\mathbf{M}\mathbf{C}_a^{(n+1)} = \mathbf{M}\mathbf{C}_a^{(n)} + \Delta t\, \mathbf{K}_M \left(\boldsymbol{\mu}_b^{(n)} - \mathbf{K}_\varepsilon \mathbf{C}_a^{(n)}\right) + \Delta t\, \mathbf{r}_a^{(n)}} \tag{29}$$

Right-hand side construction

```
Grad_EM.Mult(CpV0, Lp1);
MuV += Lp1;
```

```
Grad_MM.Mult(MuV, Lp2);
Lp2.Neg();
Lp2 *= Constants::dt;

Fcb *= Constants::dt;
Lp2 += Fcb;

MmatCH.Mult(CpV0, RHCp);
RHCp += Lp2;
```

So that the complete right-hand side becomes

$$\boxed{\mathbf{b} \equiv RHCp = M\mathbf{C}^n + Lp2 + \Delta t\,\mathbf{F}}$$

Solving the system

The linear system

$$\mathbf{M}\mathbf{C}^{n+1} = \mathbf{b}$$

is solved via

```
MCH_solver.SetOperator(*MmatCH);
MCH_solver.Mult(RHCp, CpV0);
Cn.Distribute(CpV0);
```

$$\boxed{\texttt{MCH_solver.Mult(RHCp, CpV0)}}$$

This line advances the anode concentration by one timestep.

2 Electropotential Governing Equations

2.1 Cathode Electropotential (Smoothed Boundary)

Electronic transport in the cathode is governed by charge conservation within the cathode and by electrochemical reactions occurring at the cathode-electrolyte interface. In the smoothed boundary method (SBM), the cathode indicator function ψ_c restricts conduction to the particle, while interfacial reactions appear as volumetric source terms proportional to $|\nabla\psi_c|$.

2.1.1 Strong form with smoothed boundary weighting

The electropotential in the cathode ϕ_c is:

$$\nabla \cdot \big(\psi_c \kappa_c \nabla \phi_c\big) \; - \; |\nabla \psi_c| \, z_- F \, r_c = 0 \tag{30}$$

where κ_c is the electronic conductivity of the cathode, z_- is the charge number of lithium ions, F is Faraday's constant, and r_c is the Butler–Volmer reaction rate at the cathode–electrolyte interface.

The second term represents charge transfer between the cathode and electrolyte due to interfacial electrochemical reactions.

2.1.2 Weak form

Multiplying Eq. (30) by a test function v and integrating over the domain Ω gives

$$\int_\Omega v \nabla \cdot (\psi_c \kappa_c \nabla \phi_c) \, d\Omega - \int_\Omega v |\nabla \psi_c| z_- F r_c \, d\Omega = 0 \tag{31}$$

Applying integration by parts to the conduction term gives

$$\int_\Omega \psi_c \kappa_c \nabla \phi_c \cdot \nabla v \, d\Omega = \int_\Omega v |\nabla \psi_c| z_- F r_c \, d\Omega \tag{32}$$

2.1.3 Final weak form

The weak form solved in BESFEM for the cathode electropotential is

$$\boxed{\int_\Omega \psi_c \kappa_c \nabla \phi_c \cdot \nabla v \, d\Omega = \int_\Omega v |\nabla \psi_c| z_- F r_c \, d\Omega} \tag{33}$$

2.1.4 Finite-element discretization

Using basis functions $\{\phi_i\}$:

$$K\boldsymbol{\phi} = \mathbf{F} \tag{34}$$

with

$$K_{ij} = \int_\Omega \psi_c \kappa_c \nabla \phi_i \cdot \nabla \phi_j \, d\Omega \tag{35}$$

$$F_i = \int_\Omega \phi_i |\nabla \psi_c| z_Fr_c \, d\Omega \tag{36}$$

2.1.5 K and F in the code (PotC)

Stiffness matrix K

The stiffness matrix

$$K_{ij} = \int_\Omega \psi_c \kappa_c \nabla \phi_i \cdot \nabla \phi_j \, d\Omega$$

represents electronic conduction in the cathode.

In the code, the conductivity κ_c depends on the local lithium concentration and is updated every iteration:

```
cKp = mfem::GridFunctionCoefficient(&kap);

void PotC::ParticleConductivity(mfem::ParGridFunction &Cn,
                                mfem::ParGridFunction &psx)
{
    for (int vi = 0; vi < nV; vi++){
        kap(vi) = psx(vi) * (0.01929
           + 0.7045 * tanh(2.399 * Cn(vi))
           - 0.7238 * tanh(2.412 * Cn(vi)) - 4.2106e-6);
    }
}
```

The conductivity field is wrapped as a coefficient and used to assemble the stiffness matrix:

```
fem.InitializeStiffnessMatrix(cKp, Kp2);
fem.FormLinearSystem(Kp2, ess_tdof_list_e, potential, B1t, KmP, X1v, B1v);
```

This produces the stiffness matrix

$$\boxed{K \equiv \texttt{KmP}}$$

Reaction vector **F**

The reaction term

$$F_i = \int_\Omega \phi_i |\nabla \psi_c| z_Fr_c \, d\Omega$$

accounts for electronic current at the cathode-electrolyte interface.

In the code, the reaction rate is scaled by Faraday's constant:

```
RpP = Rx;
RpP *= Constants::Frd;
```

The vector is assembled via a linear form:

```
fem.InitializeForceTerm(cRp, Bp2);
Bp2->Assemble();
Fpt = *Bp2;
```

This produces the vector:

$\boxed{\mathbf{F} \equiv \texttt{Fpt}}$ and after elimination via `FormLinearSystem`: $\boxed{\mathbf{F}_{\text{elim}} \equiv \texttt{Fpb}}$

2.1.6 Solving the Linear System

After assembling the stiffness matrix and source vector, the linear system

$$\boxed{K\phi_c = \mathbf{F}} \tag{37}$$

is formed and solved.

System assembly

```
fem.FormLinearSystem(Kp2, ess_tdof_list_e, phx, Fpt, KmP, X1v, Fpb);
```

Solver and preconditioner

The resulting system is symmetric and positive definite. It is solved using a conjugate-gradient method with an algebraic multigrid (AMG) preconditioner:

```
Mpp = std::make_unique<mfem::HypreBoomerAMG>(KmP);
cgPP_solver.SetPreconditioner(*Mpp);
cgPP_solver.SetOperator(KmP);
```

Solving the linear system

```
cgPP_solver.Mult(Fpb, Xs0);
phx.Distribute(Xs0);
```

Thus, the line

`cgPP_solver.Mult(Fpb, Xs0)`

computes the cathode electropotential satisfying Eq. (33).

2.2 Electrolyte Electropotential (Smoothed Boundary)

The electrolyte electropotential governs ionic current transport within the electrolyte phase and ensures charge conservation under coupled diffusion, migration, and electrochemical reactions. In the smoothed boundary method (SBM), the electrolyte indicator function ψ_e restricts transport to the electrolyte region, while reactions at the cathode-electrolyte and anode-electrolyte interfaces appear as volumetric source terms proportional to $|\nabla\psi_c|$ and $|\nabla\psi_a|$.

2.2.1 Strong form with smoothed boundary weighting

The electropotential in the electrolyte ϕ_e is:

$$\nabla\cdot\big[\psi_e\big(z_+m_+ - z_-m_-\big)F\,C_e\nabla\phi_e\big] + |\nabla\psi_c|\,\frac{r_c}{\nu_+} + |\nabla\psi_a|\,\frac{r_a}{\nu_+} = \nabla\cdot\big[\psi_e\big(D_- - D_+\big)\nabla C_e\big] \tag{38}$$

Here, $m_\pm$ and $D_\pm$ are the transport mobilities and diffusivities of anions and cations, respectively, $z_\pm$ are the charge numbers, F is Faraday's constant, and ν_+ is the dissolution number.

2.2.2 Weak form

Multiplying Eq. (38) by a test function v and integrating over Ω yields

$$\begin{aligned}\int_\Omega v\,\nabla\cdot[\psi_e(z_+m_+ - z_-m_-)FC_e\nabla\phi_e]\;d\Omega + \int_\Omega v\left(|\nabla\psi_c|\frac{r_c}{\nu_+} + |\nabla\psi_a|\frac{r_a}{\nu_+}\right)\,d\Omega \\ = \int_\Omega v\,\nabla\cdot[\psi_e(D_- - D_+)\nabla C_e]\;d\Omega\end{aligned} \tag{39}$$

2.2.3 Final weak form

After integration by parts, the weak form solved in BESFEM for the electrolyte electropotential is

$$\boxed{\int_\Omega \psi_e\kappa_e(C_e)\nabla\phi_e\cdot\nabla v\,d\Omega = \int_\Omega \psi_e\chi_e(C_e)\nabla C_e\cdot\nabla v\,d\Omega + \int_\Omega v\left(|\nabla\psi_c|\frac{r_c}{\nu_+} + |\nabla\psi_a|\frac{r_a}{\nu_+}\right)\,d\Omega} \tag{40}$$

where

$$\kappa_e(C_e) = (z_+m_+ - z_-m_-)FC_e, \qquad \chi_e(C_e) = D_- - D_+.$$

2.2.4 Finite-element discretization

Using basis functions $\{\phi_i\}$:

$$K_\phi\boldsymbol{\phi}_e = K_c\mathbf{C}_e + \mathbf{F} \tag{41}$$

with

$$(K_\phi)_{ij} = \int_\Omega \psi_e\kappa_e(C_e)\nabla\phi_i\cdot\nabla\phi_j\,d\Omega \tag{42}$$

$$(K_c)_{ij} = \int_\Omega \psi_e \chi_e(C_e) \nabla\phi_i \cdot \nabla\phi_j \, d\Omega \tag{43}$$

$$F_i = \int_\Omega \phi_i \left(|\nabla\psi_c| \frac{r_c}{\nu_+} + |\nabla\psi_a| \frac{r_a}{\nu_+} \right) d\Omega \tag{44}$$

2.2.5 K_ϕ, K_c, and F in the code (PotE)

Ionic conductivity matrix K_ϕ

The matrix

$$(K_\phi)_{ij} = \int_\Omega \psi_e \kappa_e(C_e) \nabla\phi_i \cdot \nabla\phi_j \, d\Omega$$

represents ionic migration driven by the electrolyte potential.

In the code, the electrolyte conductivity κ_e is concentration dependent and assembled as

```
dffe = exp(-7.02 - 830 * Cn(vi) + 50000 * Cn(vi) * Cn(vi));
kpl(vi) = psx(vi) * tc2 * Constants::D0 * dffe * Cn(vi);
```

The conductivity field is wrapped as a coefficient and used to assemble the stiffness matrix:

```
cKe.SetGridFunction(&kpl);
fem.InitializeStiffnessMatrix(cKe, Kl2);
fem.FormLinearSystem(Kl2, ess_tdof_potE, potential, Flt, Kml, X1v, Flb);
```

$$\boxed{K_\phi \equiv \texttt{Kml}}$$

Diffusion-induced current matrix K_c

The term

$$\int_\Omega \psi_e \chi_e(C_e) \nabla C_e \cdot \nabla v \, d\Omega$$

accounts for charge flux driven by concentration gradients.

In the code, the diffusivity coefficient is

```
dffe = exp(-7.02 - 830 * Cn(vi) + 50000 * Cn(vi) * Cn(vi));
Dmp(vi) = psx(vi) * tc1 * Constants::D0 * dffe;
```

and the corresponding stiffness matrix is assembled as

```
cDm.SetGridFunction(&Dmp);
fem.InitializeStiffnessMatrix(cDm, Kl1);
fem.FormLinearSystem(Kl1, boundary_dofs, potential, B1t, Kdm, X1v, B1v);
```

$$\boxed{K_c \equiv \texttt{Kdm}}$$

The product $K_c \mathbf{C}_e$ is evaluated as

```
Cn.GetTrueDofs(CeVn);
Kdm.Mult(CeVn, LpCe);
```

***Reaction vector* F**

The reaction term is

$$F_i = \int_\Omega \phi_i \left(|\nabla \psi_c| \frac{r_c}{\nu_+} + |\nabla \psi_a| \frac{r_a}{\nu_+} \right) d\Omega$$

In the code, cathode and anode reaction rates are combined:

```
RpE = Rx2;
RpE += Rx1;
RpE.Neg();
```

The source vector is assembled via

```
fem.InitializeForceTerm(cRe, Bl2);
Bl2->Assemble();
Flt = *Bl2;
```

This produces the vector:

$\boxed{\mathbf{F} \equiv \texttt{Flt}}$ and after elimination via `FormLinearSystem`: $\boxed{\mathbf{F}_{\text{elim}} \equiv \texttt{Flb}}$

2.2.6 Solving the Linear System

Collecting all parts, the linear system solved for the electrolyte electropotential is

$$\boxed{K_\phi \boldsymbol{\phi}_e = \mathbf{F} + K_c \mathbf{C}_e} \tag{45}$$

Right-hand side construction

```
RHSl = Flb;
RHSl += LpCe;
```

Solving the linear system

```
pE0 = phx;
pE0.GetTrueDofs(Xe0);
cgPE_solver.SetOperator(Kml);
cgPE_solver.Mult(RHSl, Xe0);
phx.Distribute(Xe0);
```

Thus, the line

`cgPE_solver.Mult(RHSl, Xe0)`

computes the electrolyte electropotential satisfying Eq. (40).

2.3 Anode Electropotential (Smoothed Boundary)

Electronic transport in the anode is governed by charge conservation within the anode and by electrochemical reactions occurring at the anode-electrolyte interface. Within the smoothed boundary method (SBM), the anode indicator function ψ_a restricts electronic conduction to the anode phase, while interfacial reactions appear as volumetric source terms proportional to $|\nabla\psi_a|$.

2.3.1 Strong form with smoothed boundary weighting

The anode electropotential ϕ_a satisfies

$$\nabla \cdot \big(\psi_a \kappa_a \nabla \phi_a\big) \; - \; |\nabla \psi_a| \, z_- F \, r_a = 0 \tag{46}$$

where κ_a is the electronic conductivity of the anode, z_- is the ionic charge number of lithium, F is Faraday's constant, and r_a is the Butler-Volmer reaction rate at the anode-electrolyte interface.

The second term represents electronic current transfer between the anode and electrolyte due to electrochemical reactions.

2.3.2 Weak form

Multiplying Eq. (46) by a test function v and integrating over the domain gives

$$\int_\Omega v \nabla \cdot (\psi_a \kappa_a \nabla \phi_a) \, d\Omega - \int_\Omega v |\nabla \psi_a| z_- F r_a \, d\Omega = 0 \tag{47}$$

Applying integration by parts to the conduction term gives

$$\int_\Omega \psi_a \kappa_a \nabla \phi_a \cdot \nabla v \, d\Omega = \int_\Omega v |\nabla \psi_a| z_- F r_a \, d\Omega \tag{48}$$

2.3.3 Final weak form

The weak form solved in BESFEM for the anode electropotential is

$$\boxed{\int_\Omega \psi_a \kappa_a \nabla \phi_a \cdot \nabla v \, d\Omega = \int_\Omega v |\nabla \psi_a| z_- F r_a \, d\Omega} \tag{49}$$

2.3.4 Finite-element discretization

Using basis functions $\{\phi_i\}$:

$$K \boldsymbol{\phi}_a = \mathbf{F} \tag{50}$$

with

$$K_{ij} = \int_\Omega \psi_a \kappa_a \nabla \phi_i \cdot \nabla \phi_j \, d\Omega, \tag{51}$$

$$F_i = \int_\Omega \phi_i |\nabla \psi_a| z_- F r_a \, d\Omega. \tag{52}$$

2.3.5 K and F in the code (PotA)

Stiffness matrix K

The stiffness matrix

$$K_{ij} = \int_\Omega \psi_a \kappa_a \nabla\phi_i \cdot \nabla\phi_j \, d\Omega$$

represents electronic conduction within the anode.

In the code, the anode conductivity κ_a is initialized using the smoothed indicator:

```
cKp = mfem::GridFunctionCoefficient(&kap);
kap = psx;
```

This conductivity field is wrapped as an MFEM coefficient and used to assemble the stiffness matrix:

```
fem.InitializeStiffnessMatrix(cKp, Kp2);
fem.FormLinearSystem(Kp2, ess_tdof_list_w, ph, B1t, KmP, X1v, B1v);
```

This produces the stiffness matrix

$$\boxed{K \equiv \texttt{KmP}}$$

Reaction vector $\mathbf{F}$

The reaction term

$$F_i = \int_\Omega \phi_i |\nabla\psi_a| z_Fr_a \, d\Omega$$

represents electronic current due to interfacial electrochemical reactions.

In the code, the reaction rate is scaled by Faraday's constant:

```
RpP = Rx;
RpP *= Constants::Frd;
```

The corresponding linear form is assembled as

```
fem.InitializeForceTerm(cRp, Bp2);
Bp2->Assemble();
Fpt = *Bp2;
```

This produces the vector

$$\boxed{\mathbf{F} \equiv \texttt{Fpt}} \qquad \text{and after elimination via } \texttt{FormLinearSystem} \qquad \boxed{\mathbf{F}_{\text{elim}} \equiv \texttt{Fpb}}$$

2.3.6 Solving the Linear System

After assembling the stiffness matrix and reaction source vector, the linear system

$$\boxed{K\boldsymbol{\phi}_a = \mathbf{F}} \tag{53}$$

is formed and solved.

System assembly

```
fem.FormLinearSystem(Kp2, ess_tdof_list_w, ph, Fpt, KmP, X1v, Fpb);
```

Solver and preconditioner

The system is symmetric and positive definite. It is solved using a conjugate gradient method with an algebraic multigrid (AMG) preconditioner:

```
Mpp = std::make_unique<mfem::HypreBoomerAMG>(KmP);
cgPP_solver.SetPreconditioner(*Mpp);
cgPP_solver.SetOperator(KmP);
```

Solving the linear system

```
cgPP_solver.Mult(Fpb, Xs0);
phx.Distribute(Xs0);
```

Thus, the line

```
cgPP_solver.Mult(Fpb, Xs0)
```

computes the anode electropotential satisfying Eq. (49).

3 Butler Volmer Reactions

$$r_c = k_c^f \, C_+ \exp\left(\frac{-\alpha \, z_+ F \left[\phi\right]_e^c}{RT}\right) \; - \; k_c^b \, C_c \exp\left(\frac{(1-\alpha) \, z_+ F \left[\phi\right]_e^c}{RT}\right) \tag{54}$$

$$r_a = k_a^f \, C_+ \exp\left(\frac{-\alpha \, z_+ F \left[\phi\right]_e^a}{RT}\right) \; - \; k_a^b \, C_a \exp\left(\frac{(1-\alpha) \, z_+ F \left[\phi\right]_e^a}{RT}\right) \tag{55}$$